

CHAPTER 1

INTRODUCTION

Misplaced belongings are an everyday reality in schools and universities. Students and employees move constantly between classrooms, libraries, laboratories, cafeterias, and open spaces, and personal property is frequently left behind. Campus lost-and-found units are expected to reconnect owners with recovered items, yet many institutions still depend on handwritten logs, verbal claims, and informal notices. Under those conditions, records become uneven, follow-ups are slow, and ownership is difficult to establish with confidence. Prior observations of university lost-and-found practice point to weak documentation standards and frequent misidentification as major reasons recovery remains unreliable (Mullins & Lee, 2017).

Campus studies likewise show that when formal channels are weak, people improvise. Learners and staff often announce losses on social media or group chats, but those posts are easy to miss, hard to search later, and poorly suited to verifying who should receive an item (Tan & Chong, 2023). Manual office logs create a different set of problems: inconsistent entries, exposed personal details, and slow checking when someone arrives to claim property (Castro et al., 2022). Taken together, the evidence suggests that lost-and-found work needs both better organization and a more dependable way to compare submissions.

Digitizing forms is only a partial answer. Web and mobile portals can collect reports into categories and searchable lists (Kim et al., 2019; Gupta & Sharma, 2020), yet many still leave the actual pairing of lost and found records to staff judgment or simple keyword filters. Keyword search breaks down when two people describe the same object differently, omit details, or rely mainly on photographs. Multimodal learning offers a stronger alternative. Models trained to relate images and language can place photos and captions in one representation space, then score how closely two submissions appear to refer to the same object (Peng, 2025). Architectures in the CLIP family, including Vision Transformer image encoders, make that comparison practical for service systems that must handle mixed visual and textual evidence.

This study therefore develops **FindIt**, an AI-supported campus lost-and-found matcher built around image embeddings and semantic similarity. Members of the community file lost or found tickets through a web application. Each ticket is encoded with CLIP ViT-B/32; vectors are stored centrally; and candidate pairs are ranked by a multimodal similarity pipeline. Results that clear confidence thresholds appear as an ordered shortlist so a user can confirm the correct item even when it is not ranked first. After confirmation, the platform records the claim and emails the parties so they can arrange pickup through the university’s designated desk. The intent is to automate the costly comparison step while keeping people in control of the final decision.

1.1 Project Context

Busy campuses concentrate movement, shared facilities, and high turnover of personal property. Phones, ID cards, earphones, calculators, bags, keys, and similar items are regularly abandoned in instructional and common areas. In the usual workflow, a finder brings an object to an office; staff write a brief note; and an owner later tries to reclaim it by describing the item aloud or recognizing it on a shelf. Descriptions vary widely from one person to another, office hours constrain follow-up, and recovery rates suffer.

Informal substitutes do not close the gap. Department pages and student groups may publish photos of found items, but visibility is uneven and identity checks are often casual. Without a shared digital inventory that can compare new reports against existing ones, staff shoulder repetitive visual search while owners remain uncertain whether anything matching their loss has been turned in. The result is avoidable delay, duplicated effort, and avoidable loss of property.

FindIt responds to that campus setting as a matching-centered service platform. It centralizes ticket intake, applies vision–language embeddings to score likely correspondences, and guides users through ranked review and claim confirmation. Physical custody stays with institutional process; the software contribution is faster, more consistent identification of promising lost–found pairs.

1.2 Purpose and Description

The study aims to design and build a working lost-and-found matching algorithm, together with the application services needed to use it on campus. Where conventional practice depends on staff memory and keyword browsing, FindIt converts photographs and natural-language descriptions into embeddings and ranks likely counterparts with cosine similarity and weighted multimodal fusion.

Technically, the prototype is a browser-based client connected to a FastAPI backend and a PostgreSQL database extended with pgvector. CLIP ViT-B/32 supplies a shared encoder for images and text. A lost submission searches available found inventory; a found submission reverse-searches open lost reports. In both directions, the system returns thresholded, ordered candidates rather than a single silent decision. Users may therefore select a second- or third-ranked item when it is clearly theirs. Confirmed claims update status fields and trigger coordination email so owner and finder can complete handoff at the official desk.

The prototype is intended for controlled demonstration and evaluation: reporting flows, embedding generation, ranking behavior, claim confirmation, and notification delivery. Quality expectations emphasize correct core functions, stable operation under ordinary use, and an interface that students and staff can operate without specialized training.

1.3 Objectives of the Study

1.3.1 General Objective

To develop an AI-powered lost-and-found matching algorithm that uses image embedding generation and semantic similarity analysis to automate and improve item retrieval in institutional lost-and-found operations.

1.3.2 Specific Objectives

Build a ticket-based reporting facility for lost and found submissions, capturing descriptions, images, categories, locations, dates, and contact details through a web interface.

Generate visual and textual feature representations of each submission with CLIP ViT-B/32, producing normalized embeddings for cross-modal comparison.

Rank lost and found candidates through embedding similarity methods, including cosine scoring, adaptive multimodal fusion, soft category, location, and date adjustments, confidence tiers, and ordered shortlist presentation.

Improve operational recovery support by automating comparison and by coordinating confirmed claims through status updates and email notices to the involved parties.

1.4 Conceptual Framework

FindIt is framed as an Input–Process–Output system focused on multimodal matching and claim coordination.

Inputs include account credentials for optional authenticated use; lost-ticket fields such as description, optional photo, category, place, date, and owner email; found-ticket fields such as required photo, optional description, category, place, date, and finder email; and claim actions that identify which ranked pair the user accepts.

Processes cover media validation and storage; CLIP encoding of images and text into 512-dimensional unit vectors; computation of available similarity signals; weighted fusion into a final score; tier assignment and filtering; ranking and shortlist trimming; authentication with JWT and email verification for registered users; persistence of tickets, vectors, and claims; and outbound messaging for claim lifecycle events.

Outputs include stored tickets and embeddings, ranked match lists with scores and tiers, claim records, coordination emails after confirmation, and administrative visibility into queue activity. User

confirmation and cancellation supply feedback that prevents premature closure of inventory when a suggested pair is wrong.

1.5 Scope and Limitation of the Study

1.5.1 Scope of the Study

The study covers design and implementation of FindIt as a web platform for institutional lost-and-found matching. In scope are ticket submission, CLIP-based embedding generation, multimodal similarity ranking with thresholds, ranked candidate review, claim confirmation, and email-supported coordination of pickup through campus desk procedures. Evaluation concerns functional behavior of these components as an integrated prototype. Users access the system through standard browsers over an internet connection. The software boundary includes the frontend, API, database with vector storage, matching services, and mail delivery configuration used for demonstration.

1.5.2 Limitations of the Study

Matching quality tracks the clarity of uploaded photos and the usefulness of written descriptions; dark, blurred, occluded, or highly generic items remain difficult. The encoder is a pretrained CLIP model, so performance reflects that model's representational limits and the variability of campus photos. Ranked retrieval can still surface incorrect candidates, which is why confirmation is required before a claim is treated as established. Prototype email may be written to a local outbox when SMTP is not configured. Questions of legal ownership, dispute resolution, and day-to-day physical storage policy remain with the institution rather than with the matching algorithm.

1.6 Significance of the Study

FindIt matters because it targets a routine campus service that is still largely manual at the comparison stage. An embedding-based ranker gives students and employees a single place to file reports and inspect likely matches, then continue to pickup after confirmation. Offices gain structured tickets and machine-ordered candidates instead of relying only on memory and ad hoc browsing. For the institution, the project is a concrete example of applying vision-language AI to student services. For researchers and developers, it documents an applied pipeline for multimodal scoring, thresholded shortlists, and confirmation-centered claims in a lost-and-found setting.

1.7 Definition of Terms

FindIt — The campus lost-and-found matching platform produced in this study, including its web interface, embedding services, claim workflow, and notifications.

CLIP (Contrastive Language–Image Pre-training) — A model family that learns a joint space for images and text; this implementation uses ViT-B/32.

Image Embedding / Text Embedding — Numeric vectors produced from a photograph or description so that similarity can be computed mathematically.

Semantic Similarity Analysis — Scoring how closely two submissions appear to concern the same item, chiefly via cosine similarity on normalized embeddings.

Multimodal Matching — Use of more than one CLIP-derived signal (for example, image–image and text–image) within a single ranking score.

Confidence Tier — A label derived from the final score (strong, possible, or weak) used to keep or discard candidates.

Ranked Candidate List — The ordered shortlist shown to users after filtering, from which a non-top result may still be chosen.

Ticket-Based Reporting — Structured creation of lost or found records for centralized processing.

Claim Confirmation — Acceptance steps that bind a lost ticket to a found ticket and start coordination notices.

pgvector — PostgreSQL extension for storing embedding vectors and supporting distance-based ordering.

Recovery Coordination — Post-confirmation communication that helps parties complete institutional pickup.

Matching Accuracy — How successfully the pipeline places the correct counterpart among early ranked candidates under test conditions.

CHAPTER 2

REVIEW OF RELATED STUDIES

This chapter surveys literature and related systems that motivate FindIt’s design. The discussion moves from conventional campus practice, to digital reporting platforms, to multimodal embedding retrieval, and finally to ranking policies suited to noisy real-world tickets.

2.1 Related Literature

Conventional Campus Lost-and-Found Practice

University lost-and-found work has long depended on counter staff, paper or spreadsheet logs, and face-to-face identification. Case evidence shows that incomplete notes and inconsistent wording make later matching unreliable (Mullins & Lee, 2017). Where no authoritative channel exists, communities fall back on chats and social feeds that neither preserve searchable history nor protect against mistaken or dishonest claims (Tan & Chong, 2023). Institutional write-ups of manual offices also describe cluttered records, privacy leakage, and slow validation when claimants arrive (Castro et al., 2022; Alston, 2022). The emotional cost of losing valued property adds urgency to any redesign that can shorten time-to-match (Nadeem et al., 2022). These accounts establish the service problem FindIt inherits: recovery fails not only from missing storage space, but from weak comparison infrastructure.

Digital Reporting and Notification Platforms

Browser and smartphone applications improved intake. Systems described by Kim et al. (2019) and Gupta and Sharma (2020) organize found inventory and support category or text search, yet pairing often remains manual. AUFound illustrates a split client model—mobile reporting for students and web tools for staff—with messaging that speeds awareness even while verification stays human-led (Castro et al., 2022). Campus web builds such as those of Pandey et al. (2020) and Shrivastava et al. (2025) add dashboards, uploads, and alerts, and report operational gains over bulletin-board practice. Salman (2022) emphasizes registration and direct contact between finder and owner once a candidate is known.

Across these platforms, the durable lessons are centralized tickets, media attachments, and notifications. The recurring limitation is the matching engine itself: keyword filters and staff browsing do not scale cleanly when descriptions diverge or when photos carry most of the identity signal. FindIt keeps the ticket-and-notice pattern but inserts multimodal ranking between submission and confirmation.

Multimodal Embeddings with CLIP and Vision Transformers

Cross-modal retrieval research shows that contrastive vision–language training can align photographs with captions so cosine similarity becomes a meaningful relevance score (Peng, 2025). CLIP-style

encoders, commonly built with Vision Transformer image towers, are therefore attractive when a service must compare a found photo to a lost description, or two photos of the same object taken under different conditions. Follow-on work examines data-efficient training, robustness, benchmarking, and finer local representations (Li et al., 2022; Fang et al., 2022; Cui et al., 2022; Dong et al., 2026). The shared implication is twofold: pretrained multimodal encoders are strong starting points, and downstream systems still need task-level policy because user media are noisy and look-alike objects are common.

FindIt adopts CLIP ViT-B/32 as its encoder and then applies application-level fusion, contextual soft scoring, thresholds, and shortlist trimming. That combination treats the model as a feature generator for campus tickets rather than as an unsupervised final judge.

Ranked Retrieval and Human Confirmation

Dense retrieval practice favors returning an ordered top-k set, especially when queries are short or ambiguous (Muennighoff et al., 2022; Amur et al., 2023). Lost-and-found text is typically brief, and photographs vary in framing. A pipeline that discards weak scores, ranks survivors, and asks a person to confirm before closing records fits both information-retrieval norms and campus risk control. Metrics such as hit rate at small k and mean reciprocal rank align naturally with that design because they credit systems that surface the true item early even when rank-1 is imperfect.

2.2 Synthesis

Related work charts a path from paper logs to digital portals to multimodal retrieval. Digitization solved intake and visibility more than it solved comparison. Embedding models supply the missing comparison mechanism, provided the product layer defines thresholds, ranking, and confirmation.

FindIt occupies that product layer for universities: ticket capture, CLIP embeddings, multimodal similarity ranking, ordered candidate review, and email coordination after a claim is accepted.

2.2.1 Similarities

Prior systems and FindIt share the goals of faster recovery, clearer records, and reduced dependence on purely informal channels. They also share interest in categories, media uploads, and notifying stakeholders when progress occurs.

2.2.2 Differences

Where many campus tools stop at searchable lists, FindIt computes multimodal embedding scores and presents a thresholded ranking. Where some research demos optimize benchmark retrieval alone, FindIt embeds that retrieval inside reporting and claim workflows used for institutional handoff.

2.2.3 Limitations and Research Gaps

Gaps remain around operational multimodal matching for noisy campus tickets, around over-trust in single top-1 automatic links, and around connecting ranked AI output to accountable confirmation and notice. FindIt addresses those gaps with an end-to-end matching algorithm and claim workflow tailored to university lost-and-found practice.

CHAPTER 3

TECHNICAL REQUIREMENTS

This chapter states the hardware, software, and peopleware needed to build and operate FindIt as an AI-powered lost-and-found matching platform with web access, vector storage, and claim notifications.

3.1 System Requirements

3.1.1 Hardware

Table 1. End-User Devices

Component	Specification
Client device	Computer or mobile device with a current web browser
Display	Screen large enough for forms, photo review, and ranked results; responsive layout supported
Imaging	Camera or photo library access for uploads; well-lit, focused images preferred
Network	Reliable Wi-Fi or cellular data for API calls and media transfer

Table 2. Development and Hosting Machines

Component	Specification
CPU	Multi-core processor for concurrent API work and model inference
RAM	At least 8 GB for local stacks; 16 GB preferred when API, database, and CLIP run together
GPU	Optional CUDA device to accelerate encoding; CPU mode supported

Disk	Space for database files, uploaded images, cached weights, and mail outbox artifacts
Ports / LAN	Connectivity among frontend, API, and database services during development

3.1.2 Software

Table 3. Frontend Stack

Component	Specification
Interface	Static web application for FindIt user flows
Serving	nginx within the Docker-based development compose setup
Browsers	Recent Chrome, Firefox, Edge, or Safari
Capabilities	Registration and login, lost/found filing, match review, claims, and queue inspection

Table 4. Backend Stack

Component	Specification
Language	Python 3.x
API framework	FastAPI served with Uvicorn
Database	PostgreSQL 16
Vectors	pgvector storing 512-d embeddings
Migrations	Alembic

Security	JWT sessions; verified email required for registered sign-in
Media	Server-side upload validation, resize, and static delivery

Table 5. Matching Stack

Component	Specification
Runtime	PyTorch
Model	CLIP ViT-B/32
Vectors	L2-normalized 512-d image and text embeddings
Similarity	Cosine similarity
Policy layer	Adaptive multimodal fusion; category, location, and date adjustments; tiers; ranked trimming
Category aid	Prompt-based CLIP category suggestion

Table 6. Messaging and Deployment

Component	Specification
Mail	SMTP when configured; otherwise local mail outbox for demos
Events	Claim acceptance, cancellation, and processed notices
Containers	Docker Compose services for API, database, and frontend

3.1.3 Peopleware

Table 7. Build Roles

Role	Responsibility
Frontend developer	Web flows for reporting, ranking review, auth, and claims
Backend developer	API routes, persistence, auth, and claim state machine
Matching engineer	CLIP integration, scoring policy, and evaluation scripts
Database steward	Schema, pgvector setup, and migration hygiene
Tester	End-to-end checks of ranking, claims, and notifications

Table 8. Operating Roles

Role	Responsibility
System administrator	Service deployment, monitoring, and backup
Desk personnel	Physical custody and release after digital confirmation
Security reviewer	Access control and protection of contacts and uploads

A thesis prototype may consolidate build roles in one developer; a live campus pilot separates technical operations from desk custody.

3.2 System Architecture

FindIt is organized in layers.

The presentation layer is the browser client used to file tickets, inspect ranked matches, confirm claims, and sign in. The application layer exposes REST endpoints for auth, category prediction, lost and found intake, match payloads, and claim lifecycle actions. The matching layer runs CLIP encoding and the

scoring policy that yields tiered, trimmed rankings. The data layer keeps relational records and vectors in PostgreSQL/pgvector and stores uploaded images on the API host. The notification layer sends claim-related mail through SMTP or records it in the outbox.

End-to-end, a found ticket can reverse-match open losses, a lost ticket can rank available finds, a user can accept a candidate that is not rank-one, and confirmation can release coordination email for desk pickup. That path keeps embedding-based retrieval at the center of the architecture while preserving institutional control of physical return.